

INFOSYS SPRINGBOARD INTERNSHIP TECHNICAL DOCUMENTATION

NAME : NETHARA G

EMAIL ID : netharagengesh@gmail.com

REPO LINK : <https://github.com/NetharaG/LLM-Evaluation-System>

BATCH NO : 1

PROJECT CODE : #M-1-9

**PROJECT TITLE : DEVELOPMENT OF AI RESPONSE VALIDATION
SYSTEM WITH HALLUCINATION DETECTION
ASSISTANCE - GROUP 2**

TABLE OF CONTENTS

S. No.	CONTENT	PAGE NO.
1	PROJECT OVERVIEW	1
2	PROBLEM STATEMENT	2
3	OBJECTIVES	3
4	SYSTEM ARCHITECTURE	5
5	FOLDER STRUCTURE	10
6	INSTALLATION GUIDE	11
7	API DOCUMENTATION	14
8	DATABASE SCHEMA	17
9	RAG WORKFLOW	19
10	AGENT DESCRIPTIONS	21
11	DASHBOARD EXPLANATION	23
12	REPORT GENERATION	25
13	TESTING RESULTS	28
14	LIMITATIONS	31
15	FUTURE WORK	32
16	CONCLUSION	34

Development of AI Response Validation System with Hallucination Detection Assistance – Group 2

1. Project Overview

The Development of AI Response Validation System with Hallucination Detection Assistance is an AI-based system developed to validate and assess the quality of responses generated by Large Language Models (LLMs). The system is designed to identify whether an AI-generated response is accurate, relevant, complete, and supported by reliable reference information.

The system uses a Retrieval-Augmented Generation (RAG) approach to retrieve relevant reference information from a knowledge base. The retrieved information is then used by specialized evaluation agents to validate the generated response.

The system evaluates AI responses using four major validation dimensions:

- **Accuracy** – verifies whether the response is factually correct.
- **Relevance** – checks whether the response appropriately addresses the given question.
- **Hallucination** – identifies unsupported or fabricated information in the response.
- **Completeness** – checks whether important information is missing from the response.

The system combines the results produced by these evaluation agents to generate an overall evaluation score and verdict. It also provides an interactive dashboard for viewing evaluation results and supports batch evaluation of multiple responses through CSV files.

Reference information can be provided through a dataset or uploaded PDF documents. The system processes the reference information, generates embeddings using the all-MiniLM-L6-v2 model, and stores the resulting vectors in ChromaDB for semantic retrieval.

The application consists of a Streamlit frontend and a FastAPI backend. The frontend provides the user interface for submitting questions, AI responses, reference documents, and batch datasets. The backend manages retrieval, validation, scoring, and report generation.

Key Features

1. **AI Response Validation**
Automatically evaluates AI-generated responses using multiple quality dimensions.
2. **RAG-Based Reference Retrieval**
Retrieves relevant reference information using semantic similarity search.
3. **Hallucination Detection**
Identifies statements that are unsupported by the retrieved reference information.
4. **Multi-Agent Evaluation**
Uses separate agents for accuracy, relevance, hallucination, and completeness.

5. Overall Verdict Generation
Combines individual agent results to produce an overall score and evaluation verdict.
6. PDF-Based Evaluation
Allows reference information to be provided through uploaded PDF documents.
7. Batch Evaluation
Supports evaluating multiple question-response pairs through CSV input.
8. Interactive Dashboard
Displays individual scores, overall results, validation status, and evaluation analysis.
9. PDF Report Generation
Generates downloadable reports containing evaluation results and analysis.

Technology Stack

Component	Technology
Programming Language	Python
Frontend	Streamlit
Backend	FastAPI
Embedding Model	all-MiniLM-L6-v2
Vector Database	ChromaDB
RAG	Retrieval-Augmented Generation
Dataset Processing	Pandas
API Communication	REST API / Requests
Visualization	Streamlit dashboard
Report Generation	PDF reporting

The overall goal of the system is to provide an automated and systematic method for validating AI-generated responses, with particular emphasis on identifying hallucinated or unsupported information and providing measurable feedback on response quality.

2. Problem Statement

Large Language Models (LLMs) are increasingly used to generate answers, explanations, and other forms of information. Although these models can produce fluent and convincing responses, the generated content may sometimes contain incorrect, irrelevant, incomplete, or unsupported information.

One of the major challenges in using LLM-generated content is hallucination, where the model produces information that is not supported by the available reference material. Such responses can appear reliable to users even when they contain factual errors or fabricated statements.

Manual validation of a large number of AI-generated responses is also time-consuming and may lead to inconsistent evaluation because different evaluators may apply different standards.

Therefore, there is a need for an automated system that can systematically validate AI-generated responses against relevant reference information and provide measurable evaluation results.

The proposed Development of AI Response Validation System with Hallucination Detection Assistance addresses this problem by using a Retrieval-Augmented Generation (RAG) approach to retrieve relevant reference information and multiple specialized validation agents to assess the AI response.

The system evaluates the response based on:

- Accuracy – whether the information provided is factually correct.
- Relevance – whether the response directly addresses the question.
- Hallucination – whether the response contains unsupported or fabricated information.
- Completeness – whether important information required to answer the question is missing.

The results from these validation agents are combined to produce an overall score, verdict, and explanation, allowing users to understand not only whether a response is acceptable but also why it passed or failed validation.

Thus, the project aims to provide a reliable, automated, and structured approach for validating AI-generated responses and assisting users in identifying potential hallucinations and quality issues.

3. Objectives

The primary objective of the Development of AI Response Validation System with Hallucination Detection Assistance is to develop an automated system capable of validating AI-generated responses against relevant reference information and identifying potential quality issues.

The specific objectives of the project are:

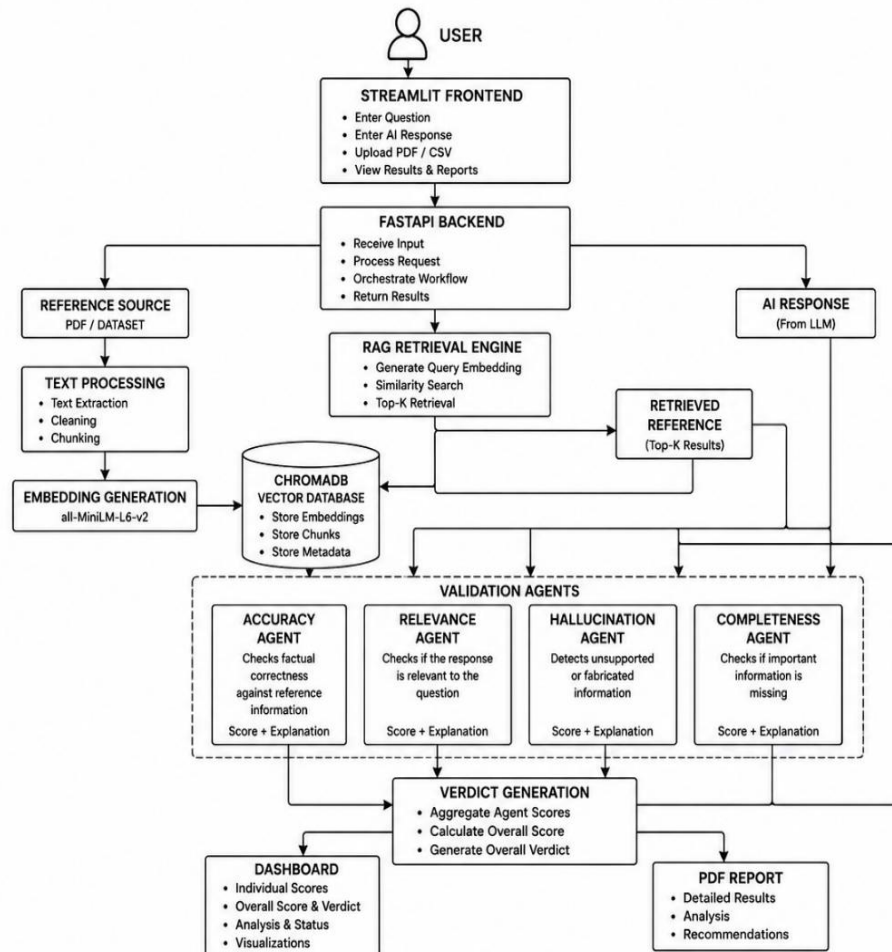
1. To develop an automated AI response validation system that evaluates responses generated by Large Language Models.
2. To implement a Retrieval-Augmented Generation (RAG) workflow for retrieving relevant reference information for the given question.

3. To generate semantic embeddings of reference information using the all-MiniLM-L6-v2 embedding model.
4. To implement vector-based information retrieval using ChromaDB for identifying relevant reference content.
5. To evaluate response accuracy by comparing the AI-generated response with the retrieved reference information.
6. To evaluate response relevance by determining whether the generated response appropriately addresses the user's question.
7. To detect hallucinated information by identifying statements that are unsupported by the available reference information.
8. To evaluate response completeness by identifying important information that may be missing from the AI-generated response.
9. To implement multiple specialized validation agents for performing independent evaluations of AI responses.
10. To generate an overall evaluation score and verdict by combining the results obtained from the individual validation agents.
11. To support PDF-based reference evaluation, allowing users to upload reference documents for response validation.
12. To support batch evaluation using CSV files, enabling multiple AI responses to be evaluated efficiently.
13. To develop an interactive dashboard for presenting validation scores, evaluation status, and analysis in a clear and understandable format.
14. To generate downloadable evaluation reports containing the results and analysis of the evaluated responses.
15. To reduce manual effort in AI response validation by providing a structured and automated evaluation workflow.
16. To provide explainable evaluation results by presenting reasons, detected hallucinated statements, missing information, and recommendations along with the scores.

4. System Architecture

The Development of AI Response Validation System with Hallucination Detection Assistance follows a modular architecture in which the frontend, backend, RAG pipeline, vector database, validation agents, and reporting components work together to validate AI-generated responses.

4.1 Architecture Overview



4.2 Frontend Layer

The frontend is implemented using Streamlit.

It provides an interactive interface through which the user can:

- enter a question
- enter an AI-generated response
- upload a reference PDF
- submit the response for evaluation
- upload CSV files for batch evaluation
- view individual validation scores

- view the overall evaluation
- examine hallucination and completeness results
- view dashboard statistics
- access generated reports

The frontend communicates with the backend through HTTP requests to the FastAPI API.

4.3 Backend Layer

The backend is implemented using FastAPI.

It acts as the central processing layer between the user interface and the evaluation components.

The backend is responsible for:

1. Receiving user input.
2. Processing uploaded reference documents.
3. Performing RAG-based reference retrieval.
4. Sending information to the validation agents.
5. Collecting agent results.
6. Generating the overall evaluation.
7. Returning the evaluation results to the frontend.
8. Processing batch evaluation requests.

The backend exposes API endpoints such as:

GET /

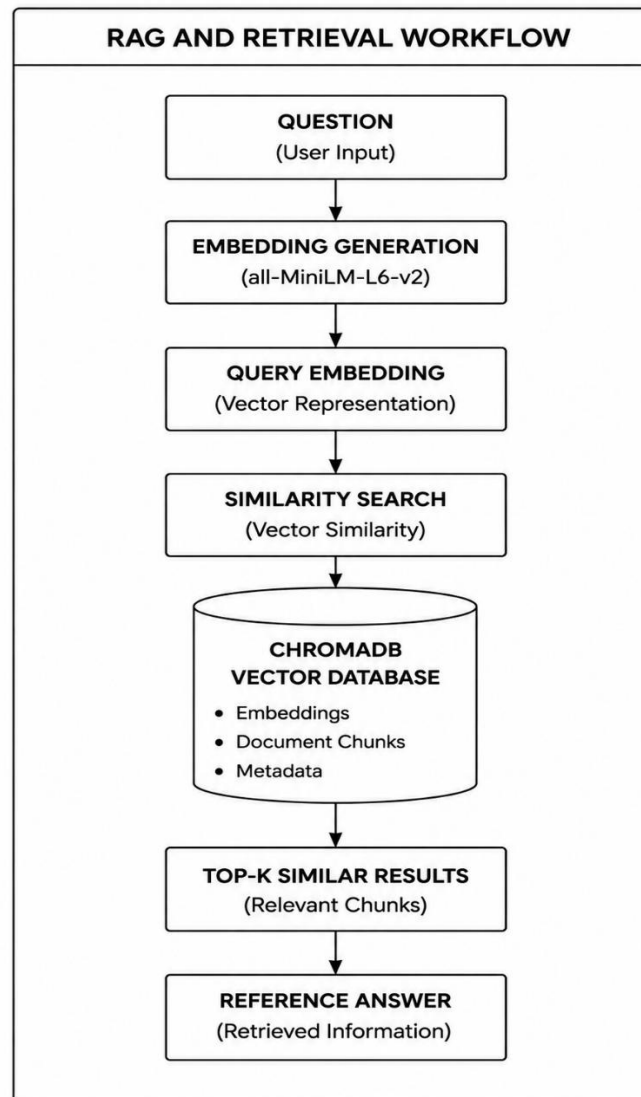
POST /evaluate

POST /batch_evaluate

4.4 RAG and Retrieval Layer

The RAG layer provides relevant reference information for validating the AI-generated response.

The process consists of:



The system uses the all-MiniLM-L6-v2 Sentence Transformer model to convert text into numerical embeddings.

The embeddings are stored in ChromaDB, which performs similarity-based retrieval when a new question is submitted.

4.5 Validation Agent Layer

After retrieving the relevant reference information, the AI response is evaluated by four specialized validation agents.

Accuracy Agent

Checks whether the information in the AI response is factually correct when compared with the reference information.

Relevance Agent

Checks whether the AI response directly addresses the question and remains relevant to the requested topic.

Hallucination Agent

Identifies information that is unsupported by the available reference information.

Completeness Agent

Checks whether important information required to answer the question is missing.

Each agent produces a score and an explanation.

4.6 Verdict Layer

The individual agent results are passed to the verdict component.

The verdict component generates:

- Overall score
- Overall verdict
- Summary of the evaluation

The validation status is represented using categories such as:

Pass

Needs Improvement

Fail

This provides the user with a consolidated assessment instead of requiring them to interpret each agent score independently.

4.7 Dashboard and Reporting Layer

The evaluation results are returned to the Streamlit frontend and displayed through an interactive dashboard.

The dashboard presents:

- Question
- AI Response
- Retrieved Reference
- Category
- Accuracy score
- Relevance score
- Hallucination score
- Completeness score
- Overall score

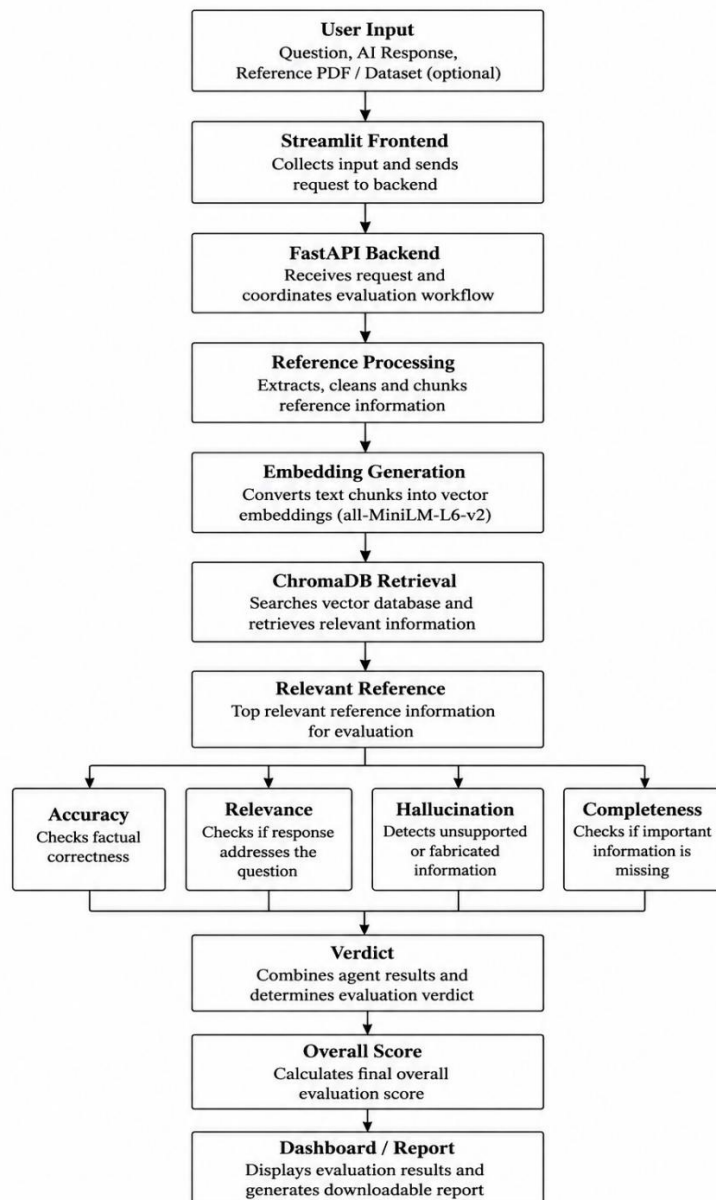
- Verdict
- Hallucination analysis
- Completeness analysis
- Validation status

For batch evaluation, the system also provides statistical visualizations and generates a downloadable PDF report.

4.8 End-to-End Data Flow

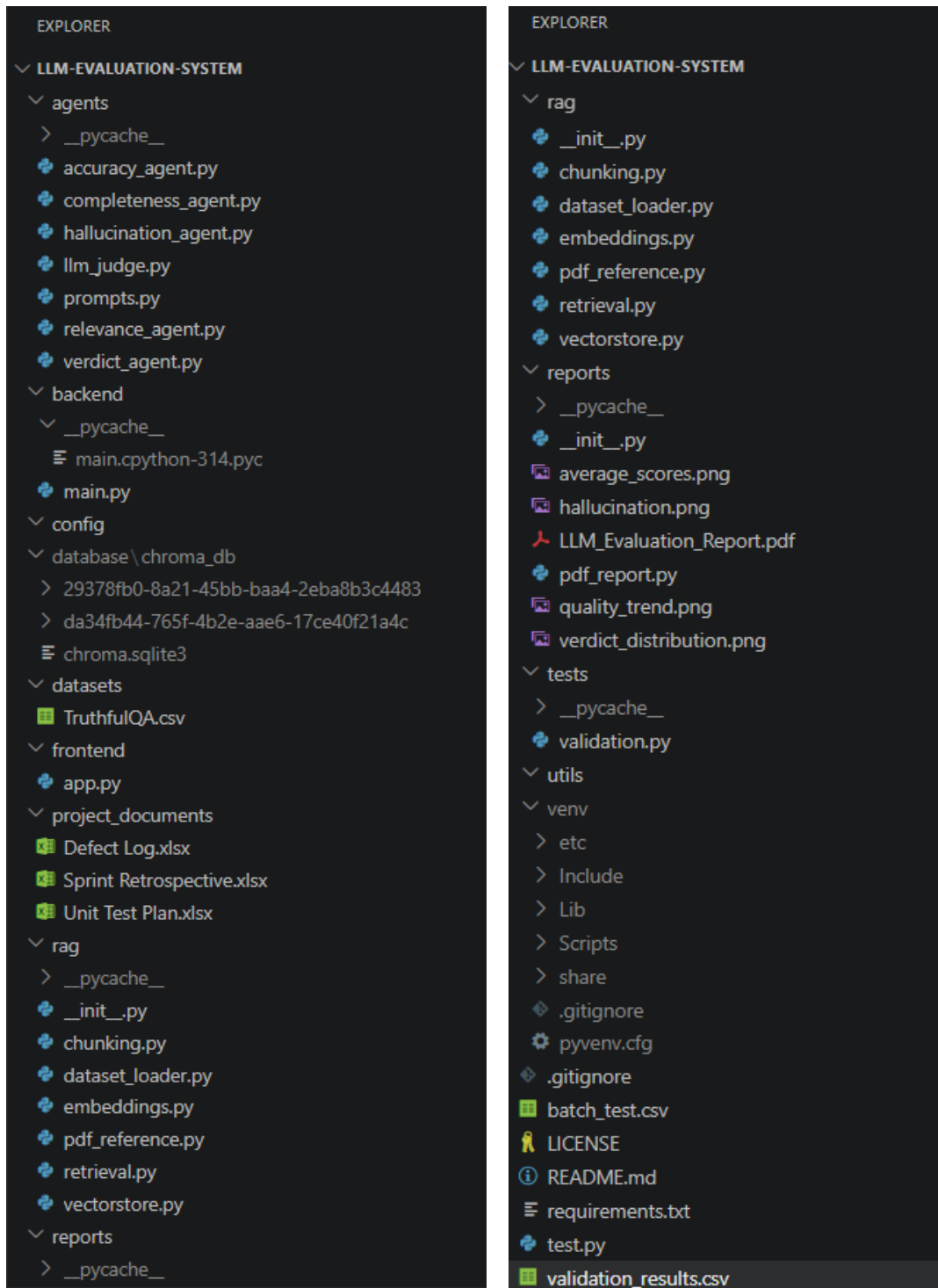
This modular architecture allows each major component to perform a specific responsibility while working together as a complete AI response validation system.

The complete system flow can be summarized as:



5. Folder Structure

The project is organized into separate folders and modules to maintain a clear separation between the frontend, backend, RAG pipeline, database, datasets, and reporting components.



5.1 Frontend

The frontend folder contains the Streamlit application. It provides the user interface for entering questions, entering AI responses, uploading reference PDFs, performing evaluations, and viewing results.

5.2 Backend

The backend folder contains the FastAPI application. It handles API requests, coordinates the evaluation workflow, performs retrieval, invokes validation agents, and returns results to the frontend.

5.3 RAG Module

The RAG-related modules handle document processing, embedding generation, and retrieval of relevant reference information.

The system uses the **all-MiniLM-L6-v2** embedding model and ChromaDB for vector-based retrieval.

5.4 Validation Agents

The agents module contains the four specialized validation agents:

- Accuracy Agent
- Relevance Agent
- Hallucination Agent
- Completeness Agent

Each agent produces an evaluation score and explanation.

5.5 Database

The database folder contains the persistent ChromaDB vector database used to store embeddings and reference information for semantic retrieval.

5.6 Datasets

The datasets folder contains reference datasets used by the system, such as the TruthfulQA dataset.

5.7 Reports

The reporting module generates evaluation reports containing the evaluation results and analysis. The system supports downloadable PDF reports.

6. Installation Guide

6.1 Prerequisites

The following software and tools are required:

- Python
- Visual Studio Code
- Git

- Streamlit
- FastAPI
- ChromaDB
- Sentence Transformers
- Pandas

The main technology stack of the project includes Python, Streamlit, FastAPI, all-MiniLM-L6-v2, ChromaDB, Pandas, REST API/Requests, and PDF reporting.

6.2 Create Virtual Environment

Open the project folder in Visual Studio Code and open the terminal.

```
python -m venv venv
```

Activate the virtual environment:

```
venv\Scripts\activate
```

6.3 Install Dependencies

Install the required Python packages:

```
pip install -r requirements.txt
```

Important packages include:

fastapi

uvicorn

streamlit

pandas

chromadb

sentence-transformers

requests

pypdf

reportlab

6.4 Run the Backend

Start the FastAPI backend using:

```
uvicorn backend.main:app --reload
```

The backend runs on:

http://127.0.0.1:8000

The project defines endpoints including GET /, POST /evaluate, and POST /batch_evaluate.

6.5 Run the Frontend

Open another terminal, activate the virtual environment, and run:

```
streamlit run frontend/app.py
```

The Streamlit interface can then be accessed through the URL displayed in the terminal.

6.6 System Execution

The general execution sequence is:

Start Backend



Start Streamlit Frontend



Enter Question



Enter AI Response



Provide Reference / Upload PDF



Click Evaluate



RAG Retrieval



Validation Agents



Overall Score & Verdict



Dashboard / PDF Report

7. API Documentation

The backend of the Development of AI Response Validation System with Hallucination Detection Assistance is implemented using FastAPI. It provides API endpoints through which the Streamlit frontend communicates with the backend for evaluation and batch processing. "The system exposes the following main endpoints: GET /, POST /evaluate, and POST /batch_evaluate."

7.1 GET /

This endpoint is used to check whether the FastAPI backend is running correctly.

Method: GET

Endpoint:

/

Purpose:

- Checks backend availability.
- Confirms that the FastAPI server is active.
- Can be used as a basic health check.

Expected Response:

```
{  
  "message": "LLM Evaluation System Backend is running"  
}
```

7.2 POST /evaluate

The /evaluate endpoint performs the main single-response evaluation.

Method: POST

Endpoint:

/evaluate

Input

The request contains:

Question

AI Response

Reference Information / Uploaded PDF

The backend receives the question and AI-generated response from the Streamlit frontend.

Processing

The endpoint performs the following operations:

1. Receives the question and AI response.
2. Processes the reference information.
3. Generates a query embedding.
4. Performs semantic retrieval using ChromaDB.
5. Retrieves relevant reference information.
6. Sends the information for validation.
7. Runs the Accuracy, Relevance, Hallucination, and Completeness evaluations.
8. Calculates the overall score.
9. Generates the final verdict.
10. Returns the results to the frontend.

Example Response

The evaluation result contains information such as:

```
{  
  "accuracy": 98,  
  "relevance": 100,  
  "hallucination": 70,  
  "completeness": 90,  
  "overall_score": 89.5,  
  "verdict": "Pass"  
}
```

The exact response fields depend on the implementation of the backend.

7.3 POST /batch_evaluate

The /batch_evaluate endpoint is used to evaluate multiple AI responses from a CSV file.

Method: POST

Endpoint:

/batch_evaluate

Input: The CSV file contains multiple question-response pairs.

Example:

question,ai_response

What is photosynthesis?,Photosynthesis is the process by which green plants use sunlight...

What is gravity?,Gravity is a force that attracts objects toward each other.

Processing

For each row, the backend:

1. Reads the question.
2. Reads the corresponding AI response.
3. Retrieves relevant reference information.
4. Performs the four validation evaluations.
5. Calculates the overall score.
6. Generates the evaluation result.
7. Stores the results for dashboard presentation and reporting.

Output

The batch evaluation provides results for multiple responses, including:

- Accuracy score
- Relevance score
- Hallucination score
- Completeness score
- Overall score
- Verdict
- Evaluation analysis

The project specifically supports batch evaluation using CSV input.

7.4 API Communication Flow

The communication between the frontend and backend can be represented as:

Streamlit Frontend



HTTP Request



FastAPI Backend



RAG Retrieval



Validation Agents



Score & Verdict Generation



JSON Response



Streamlit Dashboard

The frontend communicates with the FastAPI backend through HTTP requests.

7.5 API Summary

Endpoint	Method	Purpose
/	GET	Check backend availability
/evaluate	POST	Evaluate a single AI response
/batch_evaluate	POST	Evaluate multiple responses from CSV

8. Database Schema

The system uses ChromaDB as the vector database for storing and retrieving reference information. The reference information is converted into embeddings using the all-MiniLM-L6-v2 model and then stored in ChromaDB for semantic similarity search.

8.1 ChromaDB Collection

The project uses a ChromaDB collection named:

truthfulqa

The collection stores the information required for semantic retrieval and response validation.

8.2 Stored Data

Each record in the collection contains three major components:

1. Document

The document contains the reference answer or reference text used for evaluating an AI-generated response.

Document → Reference Answer / Reference Text

2. Embedding

The reference text is converted into a numerical vector using:

all-MiniLM-L6-v2

These embeddings allow the system to perform semantic similarity searches.

3. Metadata

Metadata provides additional information associated with the reference document.

The project retrieval implementation uses fields such as:

question

category

Therefore, the logical structure can be represented as:

ChromaDB → truthfulqa → Reference Answer | Vector Embedding | Question | Category

8.3 Database Retrieval

When a user submits a question, the question is converted into an embedding using the same embedding model.

The system then performs a similarity search against the vectors stored in ChromaDB.

User Question

↓

Embedding Generation

↓

Query Vector

↓

ChromaDB Similarity Search

↓

Top-K Results

↓

Relevant Reference

The retrieved information is then passed to the validation process for evaluating the AI-generated response.

8.4 Purpose of ChromaDB

ChromaDB is used to:

- Store reference embeddings.
- Store document/reference information.
- Store metadata.
- Perform semantic similarity search.
- Retrieve relevant reference information.
- Support the RAG-based validation workflow.

This allows the system to retrieve relevant information based on semantic meaning.

9. RAG Workflow

The system uses Retrieval-Augmented Generation (RAG) to obtain relevant reference information before evaluating the AI response. The RAG layer is responsible for providing reference information required by the validation agents.

9.1 Reference Processing

Reference information can be obtained from datasets or uploaded PDF documents. The system processes the reference content before storing it for retrieval. The project supports both dataset-based and PDF-based reference information.

The processing workflow is:

Reference PDF / Dataset



Text Extraction



Text Cleaning



Text Chunking



Embedding Generation



ChromaDB

9.2 Embedding Generation

The system uses the all-MiniLM-L6-v2 Sentence Transformer model to convert text into numerical vector representations.

These vector representations capture the semantic meaning of the reference text.

Reference Text → all-MiniLM-L6-v2 → Numerical Embedding → ChromaDB

9.3 Query Retrieval

When a question is submitted, the same embedding model converts the question into a query embedding.

The system compares this query embedding with the stored embeddings in ChromaDB and retrieves the most relevant results.

Question → Query Embedding → Similarity Search → ChromaDB → Top-K Similar Results → Relevant Reference

9.4 Reference-Based Validation

The retrieved reference information is provided to the validation agents along with the AI-generated response.

Retrieved Reference + AI Response → Validation Agents → Accuracy, Relevance, Hallucination, Completeness → Overall Score → Final Verdict

This RAG-based approach provides the validation agents with relevant reference information instead of evaluating the AI response without supporting context.

9.5 RAG Workflow Summary

The complete workflow is:

User Question



Embedding Generation



Query Embedding



Similarity Search



ChromaDB



Top-K Relevant Results



Retrieved Reference



Validation Agents

10. Agent Descriptions

The system uses four specialized validation agents to evaluate an AI-generated response from different perspectives. Each agent performs a specific validation task and produces a score along with an explanation.

10.1 Accuracy Agent

The Accuracy Agent evaluates whether the information provided in the AI response is factually correct when compared with the retrieved reference information.

Responsibilities:

- Compare the AI response with the retrieved reference.
- Identify factual errors or incorrect information.
- Determine whether the main claims are supported by the reference.
- Assign an accuracy score.
- Provide a reason for the assigned score.

Output:

- Accuracy Score
- Accuracy Reason

10.2 Relevance Agent

The Relevance Agent determines whether the AI-generated response appropriately answers the given question.

Responsibilities:

- Compare the question with the AI response.
- Check whether the response directly addresses the question.
- Identify irrelevant or unrelated information.
- Assign a relevance score.
- Provide an explanation for the evaluation.

Output:

- Relevance Score
- Relevance Reason

10.3 Hallucination Agent

The Hallucination Agent identifies information in the AI response that is unsupported by the retrieved reference.

Responsibilities:

- Compare statements in the AI response against the retrieved reference.
- Identify unsupported claims.
- Detect potentially fabricated information.
- Assign a hallucination score.
- Report the unsupported statement when hallucination is detected.

Output:

- Hallucination Score
- Hallucination Reason
- Hallucinated Statement

10.4 Completeness Agent

The Completeness Agent evaluates whether the AI response contains the important information necessary to answer the question adequately.

Responsibilities:

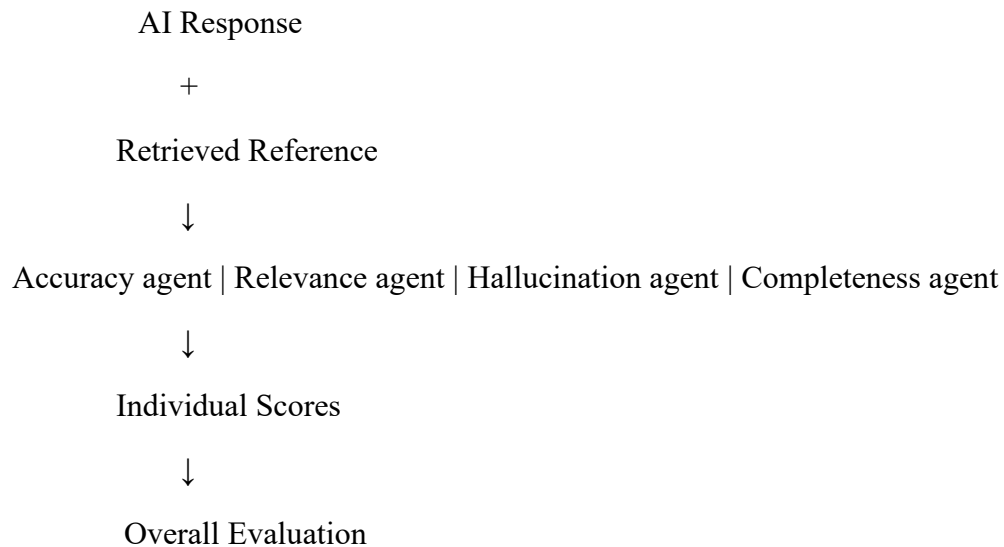
- Compare the AI response with the retrieved reference.
- Identify important information missing from the response.
- Determine whether the response provides sufficient information.
- Assign a completeness score.
- Explain the reason for the assigned score.

Output:

- Completeness Score
- Completeness Reason
- Missing Points

10.5 Agent Evaluation Process

The four agents independently evaluate the AI-generated response.



The individual results are then used by the system to produce the overall score and final verdict.

11. Dashboard Explanation

The system provides an interactive Streamlit dashboard for displaying the results of the AI response validation process.

The dashboard allows users to understand the evaluation results without having to inspect the backend response directly.

11.1 Evaluation Summary

The Evaluation Summary section displays the main information used during evaluation:

- Question
- AI Response
- Retrieved Reference
- Category

This allows the user to verify what information was provided to the validation system.

11.2 Judge Agent Evaluation

The dashboard displays the results of the four validation agents:

Agent	Evaluation
Accuracy Agent	Factual correctness
Relevance Agent	Relevance to question
Hallucination Agent	Unsupported information
Completeness Agent	Missing information

Each agent displays its score and evaluation reason.

11.3 Performance Dashboard

The Performance Dashboard presents the individual scores using metric cards.

The displayed metrics include:

- Accuracy
- Relevance
- Hallucination
- Completeness
- Overall Average

This provides a quick overview of the quality of the evaluated AI response.

11.4 Overall Evaluation

The Overall Evaluation section presents:

- Overall status
- Final verdict
- Overall score
- Evaluation summary

The overall score provides a consolidated view of the response quality based on the validation results.

11.5 Hallucination Detection

The Hallucination Detection section indicates whether unsupported information was identified.

If hallucination is detected, the dashboard displays the unsupported statement identified during evaluation.

If no important hallucination is detected, the dashboard displays a successful validation message.

11.6 Completeness Analysis

The Completeness Analysis section displays the completeness evaluation reason.

If information is missing, the system displays the identified missing points. Otherwise, the dashboard indicates that no important information is missing.

11.7 Validation Status

The Validation Status section provides individual status indicators for:

- Accuracy
- Relevance
- Hallucination
- Completeness

This allows the user to quickly identify which validation criteria have passed and which require improvement.

11.8 Backend JSON

The dashboard also provides an expandable section for viewing the raw JSON response returned by the FastAPI backend.

This is useful for:

- Debugging
- Testing
- Verifying API output
- Checking individual evaluation fields

12. Report Generation

The Report Generation module provides a structured way to record and present the results obtained from the AI response validation process. After an evaluation is completed, the system collects the validation results and prepares them for reporting.

12.1 Purpose of Report Generation

The main purpose of the reporting module is to provide a permanent and organized record of the evaluation.

The generated report can be used for:

- Reviewing evaluation results.
- Documenting AI response quality.
- Analyzing validation scores.

- Identifying hallucinated information.
- Reviewing missing information.
- Maintaining evaluation records.
- Sharing evaluation results with users or evaluators.

12.2 Information Included in the Report

The report contains important information generated during the evaluation, including:

- Question
- AI-generated response
- Retrieved reference
- Category
- Accuracy score and reason
- Relevance score and reason
- Hallucination score and reason
- Completeness score and reason
- Missing points
- Overall score
- Overall verdict
- Overall evaluation summary

12.3 Report Generation Workflow

The report generation process follows these steps:

Evaluation Completed



Collect Evaluation Results



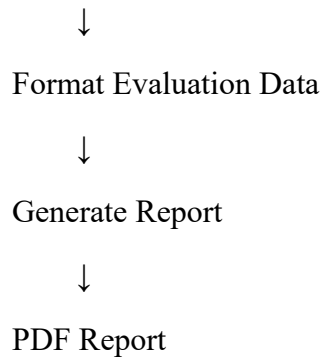
Collect Agent Scores



Collect Agent Reasons



Collect Overall Verdict



12.4 Report Structure

The generated report is organized into different sections to make the evaluation easy to understand.

Evaluation Details

This section contains the question, AI response, retrieved reference, and category.

Agent Evaluation

The results of the four validation agents are presented separately:

- Accuracy
- Relevance
- Hallucination
- Completeness

Overall Evaluation

This section presents the overall score, verdict, and summary generated from the evaluation.

Hallucination Analysis

If unsupported information is detected, the report includes the identified hallucinated or unsupported statement.

Completeness Analysis

This section provides information about missing points identified by the Completeness Agent.

12.5 PDF Report

The final evaluation can be provided as a PDF report, allowing the user to save the evaluation results for later reference.

The reporting functionality complements the Streamlit dashboard by providing a downloadable representation of the evaluation instead of requiring the user to view the results only through the application.

Report Generation Flow

Question + AI Response



RAG Retrieval



Agent Evaluation



Overall Evaluation



Report Generator



PDF Report

The report generation component therefore provides a convenient way to document and preserve the results produced by the AI Response Validation System.

13. Testing Results

Testing was performed to verify that the Development of AI Response Validation System with Hallucination Detection Assistance works correctly across its major components. The testing focused on the RAG retrieval process, single-response evaluation, PDF-based evaluation, batch evaluation, dashboard output, and report generation.

13.1 Testing Objectives

The main objectives of testing were:

- To verify that the application starts correctly.
- To verify communication between Streamlit and FastAPI.
- To verify reference retrieval using ChromaDB.
- To verify PDF reference processing.
- To verify the four validation agents.
- To verify overall score and verdict generation.
- To verify dashboard result display.
- To verify batch evaluation using CSV files.
- To verify PDF report generation.

13.2 Test Cases

Test Case	Input / Action	Expected Result	Status
TC01	Start FastAPI backend	Backend starts successfully	Passed
TC02	Open Streamlit frontend	Dashboard loads successfully	Passed
TC03	Enter question and AI response	Input accepted by application	Passed
TC04	Evaluate a single response	Evaluation result displayed	Passed
TC05	Retrieve reference from ChromaDB	Relevant reference information retrieved	Passed
TC06	Upload a PDF reference	PDF text extracted and processed	Passed
TC07	Evaluate using uploaded PDF	Response evaluated against PDF reference	Passed
TC08	Accuracy Agent evaluation	Accuracy score and reason generated	Passed
TC09	Relevance Agent evaluation	Relevance score and reason generated	Passed
TC10	Hallucination Agent evaluation	Hallucination result generated	Passed
TC11	Completeness Agent evaluation	Completeness score and missing points generated	Passed
TC12	Overall evaluation	Overall score and verdict generated	Passed
TC13	Dashboard validation status	Individual validation statuses displayed	Passed
TC14	CSV batch evaluation	Multiple responses evaluated	Passed
TC15	PDF report generation	Evaluation report generated	Passed

13.3 RAG Testing

The RAG retrieval module was tested using questions entered by the user.

The system successfully:

1. Accepted the question.
2. Generated an embedding using all-MiniLM-L6-v2.
3. Queried the ChromaDB collection.
4. Retrieved the top matching results.
5. Displayed the distance, question, category, and reference information.

The retrieval output was inspected during testing to verify that the vector database was functioning correctly.

13.4 PDF Evaluation Testing

PDF-based evaluation was also tested by uploading a reference document.

The backend successfully processed the uploaded PDF and produced logs such as:

Uploaded PDF: Photosynthesis.pdf

Processing uploaded PDF...

PDF text extracted.

Total PDF chunks: 1

The extracted reference information was then used during the evaluation process.

13.5 Single Response Testing

The single-response evaluation workflow was tested by entering a question and an AI-generated response.

The backend successfully returned an HTTP response:

POST /evaluate HTTP/1.1" 200 OK

The result was then displayed on the Streamlit dashboard.

13.6 Dashboard Testing

The dashboard was tested to verify that the following information was displayed correctly:

- Evaluation Summary
- AI Response
- Retrieved Reference
- Agent scores
- Agent reasons
- Performance metrics
- Overall evaluation
- Hallucination detection
- Completeness analysis
- Validation status
- Backend JSON

13.7 Timeout Observation

During testing, a ReadTimeout was observed during the first evaluation in some cases. A subsequent evaluation completed successfully.

This behavior was observed during testing and indicates that the initial processing may require additional time, particularly when PDF processing, embedding generation, or model initialization is involved.

The timeout was therefore recorded as a limitation/optimization point rather than a failure of the complete evaluation workflow.

13.8 Testing Result Summary

Overall, the major components of the system were successfully tested, including:

Frontend → Backend → RAG Retrieval → Validation Agents → Overall Evaluation → Dashboard → Report

The testing confirms that the implemented workflow is functional and can be used for AI response validation.

14. Limitations

Although the Development of AI Response Validation System with Hallucination Detection Assistance provides an automated approach for validating AI-generated responses, the current implementation has certain limitations.

14.1 Dependency on Reference Information

The quality of evaluation depends on the quality and relevance of the reference information available to the system. If the reference material is incomplete or does not contain sufficient information, the validation results may be limited.

14.2 Retrieval Accuracy

The RAG pipeline relies on semantic similarity to retrieve relevant reference information. In some cases, the retrieved results may not be the most appropriate information for a particular question.

14.3 Embedding Model Limitation

The system uses the all-MiniLM-L6-v2 embedding model. Although it provides efficient semantic representations, it may not capture every complex relationship or domain-specific meaning accurately.

14.4 Hallucination Detection Limitation

Hallucination detection is performed by comparing the AI response with the available reference information. Therefore, a statement may be considered unsupported when the reference source does not contain enough information to verify it.

14.5 PDF Processing Limitation

The quality of PDF-based evaluation depends on the extracted text. PDFs containing complex layouts, scanned pages, tables, or images may require additional processing for accurate extraction.

14.6 Initial Processing Time

The first evaluation may take longer because components such as the embedding model and other processing modules may need to initialize. This can occasionally result in a timeout if the configured request timeout is too short.

14.7 Limited Evaluation Dimensions

The current system primarily focuses on:

- Accuracy
- Relevance
- Hallucination
- Completeness

Other characteristics such as writing style, fluency, tone, readability, and detailed citation quality are not the primary evaluation dimensions of the current implementation.

14.8 Dataset Dependency

The quality of dataset-based evaluation depends on the questions, reference answers, categories, and other information available in the selected dataset.

14.9 Local Resource Requirements

The system uses local embedding generation and a persistent vector database. Therefore, system performance can depend on the available CPU, RAM, storage, and processing resources.

14.10 Overall Limitation

The system should be considered an **AI-assisted validation system** rather than an absolute replacement for human evaluation. Human review may still be useful for complex, ambiguous, or highly specialized responses.

15. Future Work

The system can be further improved by extending its current capabilities and optimizing the existing evaluation workflow.

15.1 Improved Retrieval

Future versions can use improved retrieval strategies such as hybrid keyword and semantic search, reranking, and better context selection to improve the relevance of retrieved references.

15.2 Advanced Hallucination Detection

Hallucination detection can be enhanced by introducing more detailed claim-level verification and stronger fact-checking mechanisms.

15.3 Support for More Document Formats

The system can be extended to support additional reference formats such as:

- DOCX
- TXT
- HTML
- JSON
- Web-based documents

15.4 Improved PDF Processing

Advanced document-processing techniques can be incorporated to handle scanned PDFs, tables, images, and complex document layouts more effectively.

15.5 Additional Evaluation Agents

Future versions can introduce additional specialized agents for evaluating:

- Fluency
- Clarity
- Tone
- Readability
- Citation quality
- Logical consistency

15.6 Improved Batch Evaluation

The batch evaluation module can be enhanced with additional filtering, sorting, comparison, and visualization options to help users analyze large evaluation datasets.

15.7 Performance Optimization

The system can be optimized to reduce initial model-loading time and improve response speed during the first evaluation.

15.8 Cloud Deployment

The application can be deployed to a cloud environment to provide remote access and support larger-scale evaluation workloads.

15.9 Enhanced Dashboard

Future dashboard versions can include advanced charts, historical evaluation tracking, comparison between multiple models, and interactive performance analytics.

15.10 Continuous Evaluation

The system can be extended to maintain evaluation history, allowing users to monitor how AI response quality changes across different models, datasets, or versions.

15.11 Human-in-the-Loop Validation

A future version can allow human evaluators to review agent decisions, provide feedback, and use that feedback to improve the validation process.

16. Conclusion

The Development of AI Response Validation System with Hallucination Detection Assistance – Group 2 was developed to provide an automated and structured approach for validating AI-generated responses.

The system integrates Streamlit, FastAPI, Retrieval-Augmented Generation (RAG), Sentence Transformers, and ChromaDB to create an end-to-end response validation workflow. Reference information is retrieved using semantic similarity and is then used to evaluate the AI-generated response.

The system uses four specialized validation agents:

- Accuracy Agent – evaluates factual correctness.
- Relevance Agent – evaluates whether the response addresses the question appropriately.
- Hallucination Agent – identifies unsupported or potentially fabricated information.
- Completeness Agent – identifies important information that may be missing.

The results from these agents are combined to generate an overall score, verdict, and detailed evaluation summary. The Streamlit dashboard provides a clear interface for viewing the evaluation results, while PDF and batch evaluation capabilities extend the system's usability.

Testing demonstrated the working integration of the major components, including reference retrieval, PDF processing, single-response evaluation, agent-based validation, dashboard visualization, batch evaluation, and report generation.

Although the system has limitations related to retrieval quality, reference dependency, processing time, and document complexity, it provides a strong foundation for automated AI response validation. Future improvements can further enhance retrieval, hallucination detection, performance, dashboard analytics, and human-in-the-loop validation.

Overall, the project demonstrates how RAG and multi-agent validation techniques can be combined to systematically assess AI-generated responses and assist in identifying accuracy, relevance, hallucination, and completeness issues.